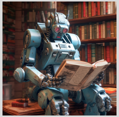
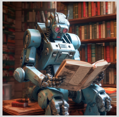


Neural Networks and Deep Learning

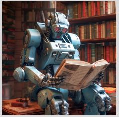
Lecture *Machine Learning* on 10.3.2026

Mario Stanke
Institut für Mathematik und Informatik
Universität Greifswald



Foundation Models

- large-scale machine learning (ML) models trained on massive datasets
- capable of performing a range of different tasks
- examples:
  - GPT, Gemini
  - Stable Diffusion, SAM (images)
  - Whisper (speech recognition)
  - models for *biological sequences*
    - protein language models (pLMs)
    - genome foundation models (gFM)



Foundation Model for Text (Informal Definition)

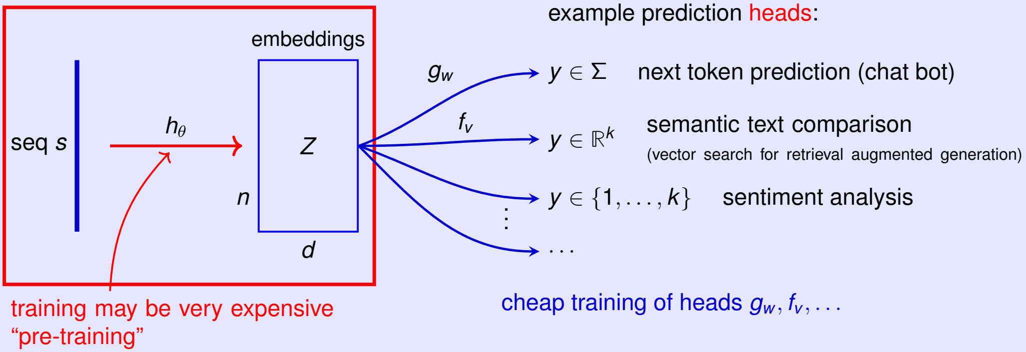
A foundation model (FM) defines an encoder function h_θ with parameters θ that maps texts (sequences of characters) $s = (s_1, \ldots, s_n) \in \Sigma^n$ of arbitrary length n to a same-length sequence of vectors:

$$(z_1, \ldots, z_n) = h_\theta(s).$$

Here, Σ is an alphabet (of characters of tokens), the *latent space dimension d* of the vectors is a model choice, the z_i are called embeddings and can be summarized in a matrix

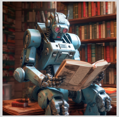
$$Z \in \mathbb{R}^{n \times d}.$$

Foundation model concept



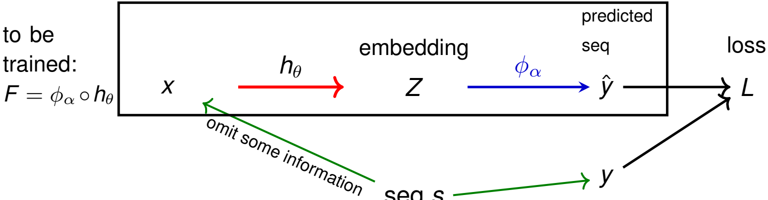
A FM is a foundation to solve diverse tasks

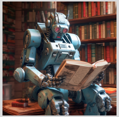
- Many machine-learning tasks are considered that require some 'understanding' of the inputs s.
- The FM h_θ learns about sequences s from a broad distribution of examples in a task-agnostic way and provides the embeddings Z that are input to other ML functions (prediction heads $g_w, f_v, \ldots$) that solve individual tasks with task-specific models and parameters.



Training Methods

- Foundation models are trained using self-supervised learning objectives.
- Supervision means that target values y for a prediction are used to estimate parameters.
- Self-supervision means that target values y and inputs x to the model are constructed from the sequences s themselves.
- Parameters θ and α are trained, i.e. chosen approximately to minimize the average loss $L = L(y, \hat{y})$ on a large training set of sequences s.
  The loss function is chosen so that L is small when y and $\hat{y}$ are similar.





1. Unidirectional / Causal / Autoregressive Modeling (GPT-style)

The model F predicts the next character in the sequence and is a model of the conditional distribution

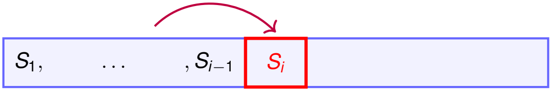
$$P(S_i \mid S_1, \ldots, S_{i-1}).$$

Example:

$$\begin{aligned} s &= \texttt{cttatgtgaaa} \\ i &= 5 \\ x &= \texttt{ctta} \\ y &= \texttt{t} \\ \hat{y} &= P(S_i = \cdot \mid (S_1, \ldots, S_{i-1}) = \texttt{ctta}) \in [0,1]^4 \end{aligned}$$

Advantages:

1. $\hat{y}$ and therefore loss values can be computed efficiently for all positions i in s in one run of a suitable model F, e.g., with so-called *causal attention*.
2. F can be directly used to sample (generate) sequences (chatbot, generative/synthetic biology).



2. Bidirectional Modeling (BERT-style)

The model F predicts characters at random positions i given the *other characters* and is a model of the conditional distribution

$$P(S_i \mid S_{-i}).$$

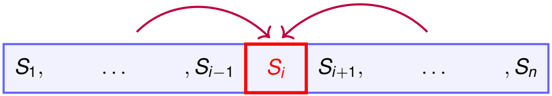
Example:

$$
\begin{aligned}
s &= \texttt{cttatgtgaaa} \\
i &= 5 \\
x &= \texttt{ctta?gtgaaa} \\
y &= \texttt{t} \\
\hat{y} &= P(S_i = \cdot \mid (S_1, \ldots, S_{i-1}) = \texttt{ctta}, (S_{i+1}, \ldots, S_n) = \texttt{gtgaaa}) \in [0, 1]^4
\end{aligned}
$$

Advantages:

1. More natural model for inherrently undirected sequences, e.g. double-stranded DNA.
2. Easier to make good predictions $\hat{y}$.

Disadvantage: No comparably efficient way known to compute loss for all positions i (multiple passes per sequence required).



Examples of Genome Foundation Models

Causal: Evo2

Nguyen et al. (2025). "Genome modeling and design across all domains of life with Evo 2."

- uses fast convolutions via Fourier transform (StripedHyena architecture)
- **Parameters:** 40B
- **Context Window:** 1Mb (Evo 2)
- **Training Data:** trillion characters from domains of life (Bacteria, Archaea, Eukaryota, Viruses), 15,032 eukaryotic genomes + > 28K prok. genomes + mitochondria, >2M bacteriophage genomes, chloroplasts, mRNA, and more, OpenGenome2
- **Training Resources:** 2,048 NVIDIA H100 GPUs for several months

Bidirectional: Nucleotide Transformer (NT)

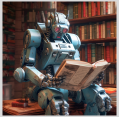
Dalla-Torre et al. (2023). "The Nucleotide Transformer: Building and Evaluating Robust Foundation Models for Human Genomics."

`https://github.com/instadeepai/nucleotide-transformer/`

- transformer architecture (BERT)
- **Parameters:** 50M to 2.5B parameters
- **Context Window:** 1 million (NT v3)
- **Training Data:** OpenGenome2 (see Evo2)
- **Training Resources:** Trained on TPU v4 pods (InstaDeep[a]/Google collaboration)

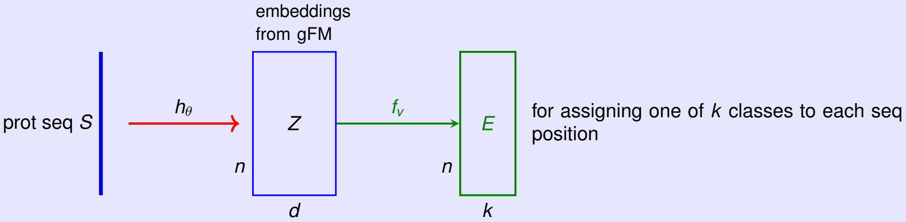
---

[a] daughter of BioNTech



Task: Positionwise Classification

Consider a head $f_v : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times k}$ (example: $d = 768$, $k = 5$)



- $E[i, j]$ could the be probability that sequence position i belongs to class j.
- Note that $E[i, \cdot]$, the i-th row of E, depends not only on character $S[i]$ but usually on the whole sequence as context.

Comparison of Training Resources

Natural Languages

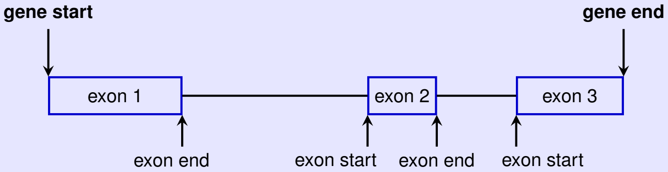
- Text data set sizes for unsupervised pre-training of LLMs, not including synthetic data.
  - Llama 4: 30 trillion
  - DeepSeek V3 14.8 trillion tokens
  - Grok3 12.8 trillion tokens
- quality is an issue
- high-quality text on the internet is approximately 15 trillion tokens
- 1 token is $\sim$ 4 characters or $\sim$3-4 bytes of information entropy
- 15 trillion tokens correspond to 45-60 TB (tera bytes)

DNA

- in December 2025 GenBank and WGS had $6651459875408 + 42125323988215 \approx 48.8$ trillion nucleotides in assembled genomes, the doubling time was 36 months
- the Sequence Read Archive (SRA) has 88.5 PB (peta bytes) in (unassembled) reads
- Biological sequences are strictly quality controlled: up to (rare) sequencing errors, they have been experimentally tested to code for a functional organism.

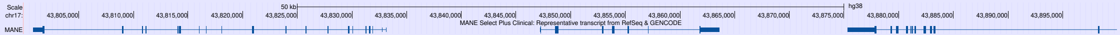
Gene Boundary Prediction

Reconsider the positionwise labeling problem from exercise set 3 exercise 0:

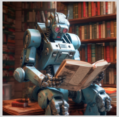


label	meaning
0	background
1	gene start
2	gene end
3	exon start
4	exon end

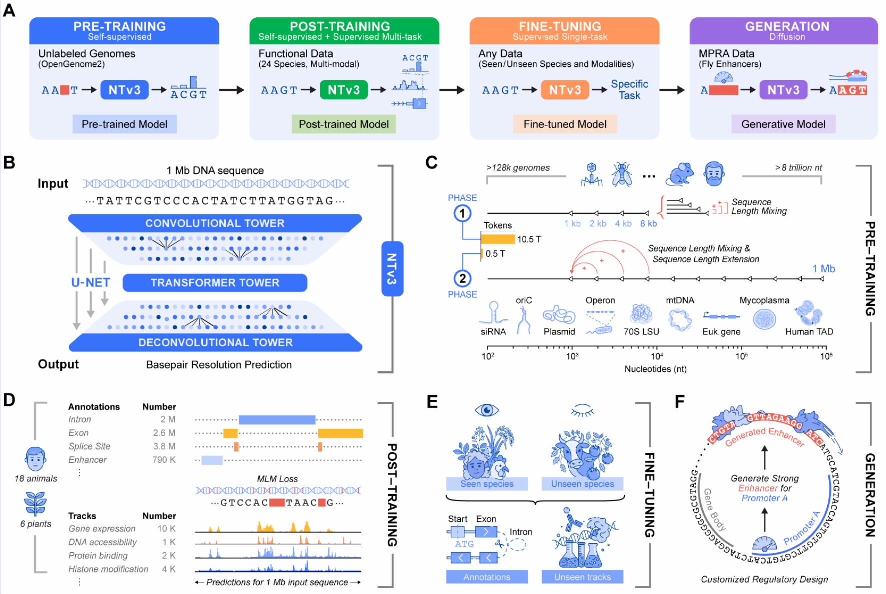
- protein-coding parts considered
- success in this task arguably necessary for per gene interpretation such as expression
- context windows of length $L = 100\,000$ for training and testing



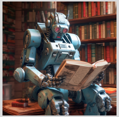
a region of length $L = 100\,000$ in the human genome with 3 genes



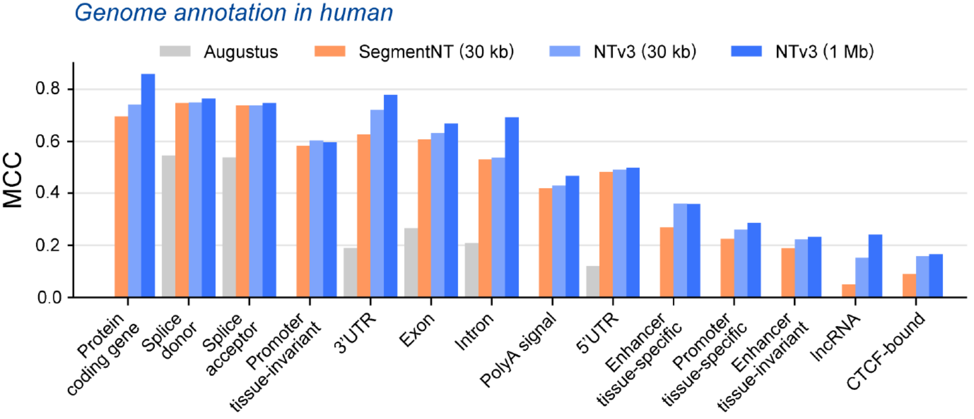
Genome Foundation Model Nucleotide Transformer v3



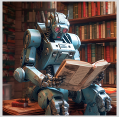
Boshar et eta, A foundational model for joint sequence-function multi-species modeling at scale for long-range genomic prediction



Genome Foundation Model Nucleotide Transformer v3



Boshar et eta, A foundational model for joint sequence-function multi-species modeling at scale for long-range genomic prediction

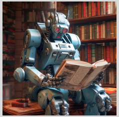


Transfer Learning: NTv3 embeddings + small CNN head

Hugging Face login + InstaDeep token required, weights downloaded, model run locally on A100

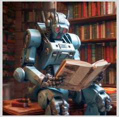
```
from transformers import AutoModelForMaskedLM, AutoTokenizer

nt_repo = "InstaDeepAI/NTv3_100M_pre"
tokenizer = AutoTokenizer.from_pretrained(nt_repo, trust_remote_code=True)
nt_model = AutoModelForMaskedLM.from_pretrained(nt_repo, trust_remote_code=True)

proj = nn.Linear(embedding_dim, 16)
conv = nn.Conv1d(16, conv_channels=8, kernel_size=3, padding="same")
head = nn.Linear(conv_channels, 5)

batch = tokenizer(sequences, # here character strings
    add_special_tokens=False, padding=True, pad_to_multiple_of=128, return_tensors="pt")

with torch.no_grad():
  out = self.nt_model( **batch, output_hidden_states=True)

embeddings = out.hidden_states[-1] # last hidden states [B,L,D], D=256, 768 or 1536
x = proj(embeddings)
x = nn.relu(x)
x = x.transpose(1, 2)
x = conv(x)
x = nn.relu(x)
x = x.transpose(1, 2)
return head(x)
```

$\Rightarrow$ Embeddings were recomputed each epoch.
What would have been their disk space usage for a human genome with 3 billion characters?



Baseline CNN

```
features = nn.Sequential(
  nn.Conv1d(4, 64, kernel_size=9, padding="same"),
  nn.ReLU(),
  nn.Conv1d(64, 64, kernel_size=9, dilation=9, padding="same"),
  nn.ReLU(),
  nn.Conv1d(64, 64, kernel_size=9, dilation=3, padding="same"),
  nn.ReLU()
)

classifier = nn.Sequential(
  nn.Linear(64, 64),
  nn.ReLU(),
  nn.Linear(64, 5)
)

h = features(x)
h = h.transpose(1, 2)
return self.classifier(h)
```



Results

model	parameters	training time transfer learning [m]	F1 [%] gene start	F1 [%] gene end	F1 [%] exon start	F1 [%] exon end
Baseline CNN	80 709	486	12.9	2.0	34.1	37.7
NTv3_8M_pre	**8M** + 4 549	736	0	0	0	0
NTv3_100M_pre	**100M** + 12 741	782	31.1	32.5	80	82.5
NTv3_100M_pre[1]	**100M** + 12 485	740	28.0	28.1	77.2	79.7
NTv3_650M_pre	**650M** + 25 029	936	34.0	46.1	78.5	86.9

Training on an HPC node with one Nvidia A100 GPU.

[1] Conv1D kernel size 1